

TP 2 – Shell

Nombre de la shell: Ysh

La shell la desarrollo con Python porque lo usé bastante más que C y me siento más cómodo. El enfoque elegido es un enfoque de Seguridad para controlar qué puede hacer el usuario. Incluí por gusto personal la posibilidad de cambiar de idioma, ya hice antes con otros programas y me parecía interesante, la shell se puede usar en español e inglés.

11 de Noviembre – Implementación del bucle REPL, pwd y cd

Implementación – Bucle REPL: Usé un bucle while True en el main, de momento sin verificaciones todavía.

Implementación – Como llamar a cada comando: Para poder hacer lo más adaptable posible, cada comando tiene como función principal una llamada “run” donde se ejecuta todo el comando al ser llamado, para hacerlo genérico uso una librería llamada pkgutil con la que itero sobre todos los archivos de la carpeta de comandos, luego con otra librería llamada importlib puedo generar objetos con las funciones “run” de cada comando iterado para llamarlos dinámicamente

Implementación – pwd: Uso getcwd del package os

Implementación – pwd: Uso chdir del package os

12 de Noviembre – Implementación de cd completo y creo el archivo “var” para las variables globales

Detalles: Decidi que cd sin argumentos va a ir directamente al directorio Home (~).

Detalles: Implementé los casos de:

- Cambio a Home
- Cambio al directorio anterior
- Cambio al directorio padre

13 de Noviembre – Implementación de ls completa

Implementación: Usé os.listdir para listar y mostrar los elementos con un loop. Decidi implementar también el argumento -a para mostrar los elementos ocultos.

Problemas: No fue un problema como tal, pero estuve un tiempo pensando que estaba mal mi función ls porque al dar como argumento “C:”, me tiraba los mismos archivos del directorio actual, no de esa ruta, había sido hace falta “C:\” para que muestre, comprobé viendo en bash que también sucede lo mismo así que dejé así eso

13 de Noviembre – Implementación de cp y rm completos

Implementación – rm: Use remove de la librería os y use los try and except para manejar las distintas excepciones posibles (Parámetro siendo un directorio, no tener permiso o no encontrar el archivo)

Implementación – cp: Leo el archivo del primer parámetro en bloques de 4096 bytes y los escribo posteriormente en el archivo destino (segundo parámetro)

15 de Noviembre – Implementación de echo y mkdir completos

Implementación – echo: Imprimo los argumentos uniendo con “ ” pero si se usa el argumento “>” se crea un archivo para guardar ahí el texto o sino se sobrescribe con open parámetro “w”.

Detalles: Usé una verificación para saber si entre los argumentos esta “>” para saber si se redirige o no, si no está simplemente uno con “ ” los argumentos

Implementación – mkdir: Usé mkdir de os e hice las verificaciones para muchos argumentos o ningún argumento

17 de Noviembre – Implementación de cat y exit completos, añadido “>>” para echo

Implementación – cat: Usé simplemente un open() para leer el archivo.

Implementación – exit: Utilizo sys.exit con argumento 0 para salir del programa.

Detalles: Añado también la posibilidad de usar “>>” para echo, así uso open pero con el argumento “a” para añadir al final del archivo

18 de Noviembre – Implementación del sistema de logging

Implementación: Creé un archivo log_gen.py que es donde se maneja la lógica de generar los registros con su función make_log

Detalles: Añadí la función make_log a todos los comandos ya implementados para registrar cada vez que se ejecuta el comando en sí, si el comando funcionó se registra en shell.log y si falló se registra en sistema_error.log, en ambos casos se colocan detalles específicos para dar más información sobre qué hicieron esos comandos.

20 de Noviembre – Personalización de la Shell con énfasis en seguridad

Implementación: Implementé el comando curf, permite manejar un sistema de “toque de queda”, de momento acepta 3 tipos de argumentos.

21 de Noviembre – Continuo con el comando curf y mejoro el parseo

Detalles: Creé el archivo curfew_utils para manejar los cargados y guardados de los archivos de persistencia

Detalles: Añadi las funcionalidades de add, rmv y list para curf, para la lista de comandos restringidos

Implementación: Uso un archivo restricted_commands.log para persistir los comandos restringidos durante el “toque de queda”

Detalles: Implementé la funcionalidad de aceptar argumentos entre comillas dobles, de esta manera todo lo que esté dentro de comillas dobles es tratado como un único argumento

9 de Diciembre – Implementación de chlan y inputlimit

Implementación – chlan: Con chlan cambio el idioma de los prints de la shell, entre español e inglés. Tuve que sustituir los prints de todo el programa por una función que recibe una key y lee del diccionario respectivo según el idioma actual para devolver el string con la respuesta.

Implementación – inputlimit:

10 de Diciembre – Correcciones finales

Problema: Al pasar a Linux la shell, me di cuenta que si bien tomé en cuenta que cambie la dirección de guardado de los archivos según sea Windows o Linux, no tome en cuenta que Linux usa "/" y Windows "\", cambié eso para que funcione en Linux

Problema: Encontré unos últimos problemas pequeños (se puede verificar con los commits cortos). Estos eran:

- curf add no guardaba en el archivo .log
- ls fallaba al dar como argumento un directorio inexistente
- curf no permitia borrar comandos ya restringidos